Class-Based SystemVerilog Verification Project

AXI4-Compliant Memory-Mapped Slave

Arsany Hany	**467**
Ziad Ahmed	**438**

GitHub Repo Link

Table of Contents

Introduction

Verifying components should be modular, reusable, and easy to maintain. It also should have clear separation and responsibilities between all its components. This lets the engineer focus on the verification intent of the project instead of testbench complexities, thus reducing wasted time.

These objectives could be achieved effectively using class-based SystemVerilog verification methods. By utilizing OOP principles, the verification environment achieves all the desired modularity, reusability, and maintainability objectives, as well as having a clear distinction between components' functionality. This is achieved through the use of separate components such as generator, driver, monitor, scoreboard, and environment. This class-based verification acts as the starting point to UVM, the standard, widely used verification methodology

This project aims to verify an AXI4 Compliant Memory-Mapped Slave IP using class-based verification methods to achieve near perfect coverage metrics, as well as detect any bugs present in the RTL design.

Findings

The RTL provided is intentionally buggy to enhance the learning of verification concepts and improve the eye-for-detail skills. Below is a list of found bugs.

- Inverted reset polarity: the reset is treated as being active-high not active-low, which does not comply with the specs.
- Memory address of by one: mem_rdata <= memory[mem_addr-1] should be mem_rdata <= memory[mem_addr].
- Boundary/validity check uses the wrong address: write_boundary_cross & read_boundary_cross are not evaluated once from the burst's start address but instead recomputed every cycle from the current, already-incremented write_addr/read_addr combined with the full original burst length. This induces double-counts and SLVERR, and causes BRESP MISMATCH.
- Missing address-alignment check: axi4.v never checks AWADDR/ARADDR against AWSIZE/ARSIZE. Causes mismatches on writes.

Coverage

Illegal and Ignored Bins

- cp_size.wrong_read_size (illegal): Any arsize other than word (3'd2). DUT only supports 32-bit transfers, so this size is never legally driven.
- cp_response.unreachable (illegal): RRESP codes 2'b01/2'b11 (reserved AXI encodings). Never architecturally producible by this DUT.
- valid_x_addr.valid_out_of_mem (ignore): Valid burst crossed with out-of-memory address region. Contradiction: an out-of-mem address makes the burst invalid by definition.
- valid_x_size.valid_nonword (ignore): Valid burst crossed with non-word arsize. Contradiction: non-word size always fails validity.
- valid_x_resp.valid_slverr (ignore): Valid burst crossed with SLVERR response. Contradiction: valid bursts always return OKAY.
- valid_x_align.valid_unaligned (ignore): Valid burst crossed with unaligned address. Contradiction: alignment is a validity precondition.
- size_x_resp.nonword_okay (ignore): Non-word size crossed with OKAY response. Contradiction: only word-sized transfers can be OKAY.

Write Coverage (axi4_write_coverage.sv)

- cp_awsize.wrong_write_size (illegal): Any awsize other than word (3'd2). Same rationale as read: DUT is 32-bit-only.
- cp_bresp.unreachable (illegal): BRESP codes 2'b01/2'b11. Reserved encodings the DUT never issues.
- cx_mode_bresp.out_of_range_okay (ignore): Out-of-range address crossed with OKAY. Contradiction: out-of-range addresses always yield SLVERR.
- cx_mode_bresp.unaligned_okay (ignore): Unaligned address crossed with OKAY. Contradiction: unaligned writes always yield SLVERR.
- cx_size_bresp.subword_okay (ignore): Non-word awsize crossed with OKAY. Contradiction: only word size can produce OKAY.
- cx_size_bresp.oversize_okay (ignore): Same expression as subword_okay (!binsof(cp_awsize.word_size) && binsof(cp_bresp.okay)), so this bin is redundant: flagging it since it doesn't add coverage exclusion beyond the previous line.

Summary & Justification

Branch coverage: 81.68%. Uncovered branches map to defensive/unreachable paths:

- The memory's else-read branch at mem_addr-1 with mem_we deasserted during reset overlap.
- The axi4 FSM's default case arms which are structurally unreachable since the state register never takes an undefined value.
- Additional misses come from read_monitor/read_driver null-mailbox check branches, which never evaluate true in a correctly wired testbench.

Condition coverage: 50.00%. The uncovered input terms are handshake conditions where one signal never independently varies against the other; e.g., AWREADY never sampled 0 while AWVALID is 1, and WVALID/WREADY never both 0 simultaneously in the required combination; plus null-handle checks (driver2gen_mbx == null) that are always false by construction and therefore never hit their true branch.

Statement coverage: 93.35%. Missing statements are inside the same unreachable default case arms and null-guard fatal calls covered above; these are defensive code paths that only execute on structurally invalid FSM states or broken mailbox wiring, neither of which occurs in a passing run.

Toggle coverage: 61.46%. Many bits never toggle because the DUT is constrained to word-only transfers (AWSIZE/ARSIZE fixed at 3'd2) as per spec, so unused size encodings and their downstream signals (write_addr_incr, read_addr_incr, write/read_size bits) never activate; RDATA/RRESP/BRESP low-order bits also stay static since only OKAY/SLVERR values ever appear.

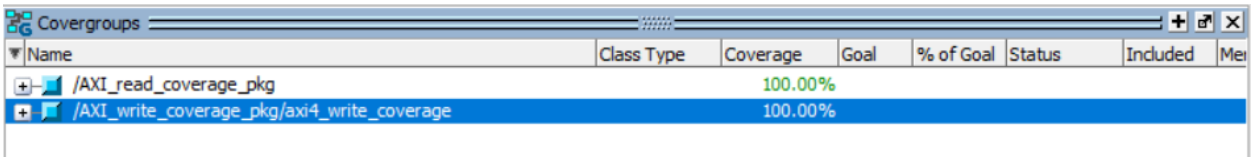
Expression/toggle-enable coverage: 72.72%. The read scoreboard's address/len/size equality expression shows 0% because expected and actual transactions always matched in this run. The false-term side of each equality was never exercised, meaning no data-mismatch scenario was driven.

Assertion coverage: 75.00%. Zero-pass assertions (a_read_address_stable, a_aw/ar_payload_stable) never fire because their antecedent requires a VALID-asserted-but-not-yet-READY stall cycle on the address channel, but the current

driver/DUT timing always completes that handshake within a single cycle, so the stall condition is never generated.

FSM coverage: 85.00% (states 100%, transitions 70-75%). The uncovered transitions (W_ADDR->W_IDLE, W_DATA->W_IDLE, R_ADDR->R_IDLE) represent a burst returning to IDLE mid-transaction; these aren't reachable through the FSM's coded behavior since a limitation exists where no test currently interrupts a burst via reset or error abort mid-flight.

QuestaSim's Covergroup



Do File

The run.do file is AI-generated to help with report organization, that is having a separate file housing each kind of coverage